

从硬件角度看内存管理：解锁计算机底层性能原理

原创 往事敬秋风 深度Linux 2025年09月01日 17:48 湖南

点击上方蓝字  免费订阅 选择 置顶公众号 

在计算机的复杂世界里，内存管理就像一位幕后指挥官，掌控着硬件性能的关键。从硬件角度深入理解内存管理，能解锁计算机底层性能的奥秘。计算机硬件犹如精密协作的交响乐团，CPU 作为核心，以惊人速度处理数据，而内存则是其紧密协作的伙伴。内存，这个计算机的“临时仓库”，承担着存储正在运行程序和数据的重任，为 CPU 提供高速数据访问通道。内存与硬盘有别，它具有易失性，断电后数据瞬间消失，但其读写速度却远非硬盘可比。像常见的动态随机存取内存（DRAM），虽需定期刷新数据，却因高存储密度和低成本成为主流；静态随机存取内存（SRAM）速度更快，不过成本高、存储密度低。

如今的双倍数据速率同步动态随机存取内存（DDR SDRAM），如 DDR4、DDR5，更是凭借高数据传输速率，大幅提升了计算机性能。在内存与 CPU 之间，还有缓存这一“秘密武器”。缓存是位于两者之间的高速存储器，容量小却速度极快，常以 L1、L2、L3 等多级形式存在。它利用程序访问数据的局部性原理，将频繁访问的数据提前存储，当 CPU 请求数据时，先在缓存中查找，若命中，数据便能迅速被获取，大大提高了程序执行效率。接下来，就让我们一同深入探索从硬件角度出发的内存管理，挖掘更多计算机底层性能的原理。



深度Linux

研究领域：Windows&Linux平台、C/C++后端开发、嵌入式和Linux系统内核等。

430篇原创内容

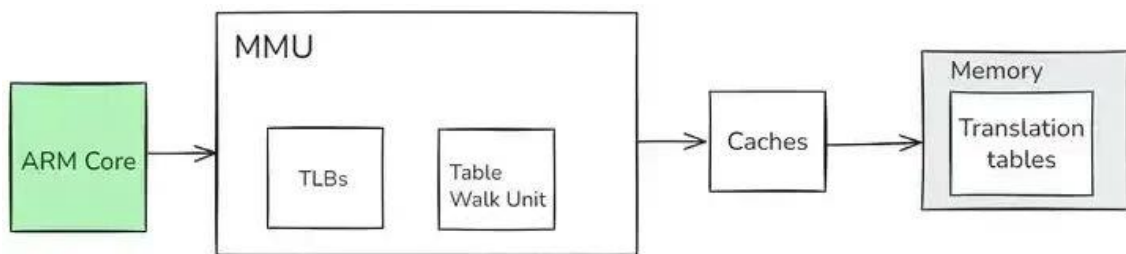
公众号

一、内存管理的硬件基石

1.1 内存管理单元（MMU）：地址转换的幕后英雄

MMU是Memory Management Unit的缩写，中文名是内存管理单元，有时称作分页内存管理单元（英语：paged memory management unit，缩写为PMMU）。它是一种负责处理中央处理器（CPU）的内存访问请求的计算机硬件。它的功能包括虚拟地址到物理地址的转换（即虚拟内存管理）、内存保护、中央处理器高速缓存的控制，在较为简单的计算机体系结构中，负责总线的仲裁以及存储体切换（bank switching，尤其是在8位的系统上）。

内存管理单元（MMU）的一个重要功能是使系统能够运行多个任务，作为独立程序在自己的私有虚拟内存空间中运行。它们不需要了解系统的物理内存映射，即硬件实际使用的地址，也不需要了解可能同时执行的其他程序。



打个比方，我们可以把计算机的内存想象成一个大型的仓库，里面存放着各种各样的物资（数据和程序）。而运行在计算机上的众多程序，就如同一个个前来领取物资的客户。如果没有一个有效的管理机制，这些客户可能会在仓库里随意翻找，不仅效率低下，还可能会出现混乱，导致物资的损坏或丢失。

而 MMU 就像是这个仓库的大管家，它制定了一套严格而有序的管理规则。每个客户（程序）在访问仓库（内存）时，都需要通过 MMU 这个大管家进行“登记”和“授权”，然后由大管家将客户的“需求指令”（虚拟地址）准确无误地转换为仓库中实际的“物资存放位置”（物理地址），这样客户就能顺利地获取到自己需要的物资，同时也保证了仓库的秩序和物资的安全。

从专业的角度来说，MMU 是一种负责处理中央处理器（CPU）的内存访问请求的计算机硬件。它的出现，让计算机系统能够更加高效、稳定地运行多个任务，仿佛为每个任务都打造了一个属于它们自己的独立小世界，互不干扰，各自精彩。

(1) MMU的核心功能

①虚拟地址与物理地址的转换魔法

在计算机的内存世界里，存在着两种重要的地址概念：虚拟地址和物理地址。虚拟地址是程序在运行时所使用的地址，就像是在地图上规划的一条虚拟路线，它并不直接对应实际的物理内存位置。而物理地址则是内存芯片上实实在在的地址，是数据真正存储的地方，就如同地图上的实际地点。

那么，MMU 是如何将虚拟地址转换为物理地址的呢？这就要借助页表这个神奇的工具了。页表就像是一本详细的地址翻译字典，记录着虚拟地址和物理地址之间的映射关系。当 CPU 发出一个虚拟地址请求时，MMU 就会像查字典一样，在页表中查找对应的物理地址。

具体来说，虚拟地址会被划分成页号和页内偏移两部分。页号就像是字典的索引，通过它可以快速定位到页表中对应的条目，而这个条目里就存储着对应的物理页号。然后，将物理页号和页内偏移组合起来，就得到了最终的物理地址，从而能够准确地访问到内存中的数据。

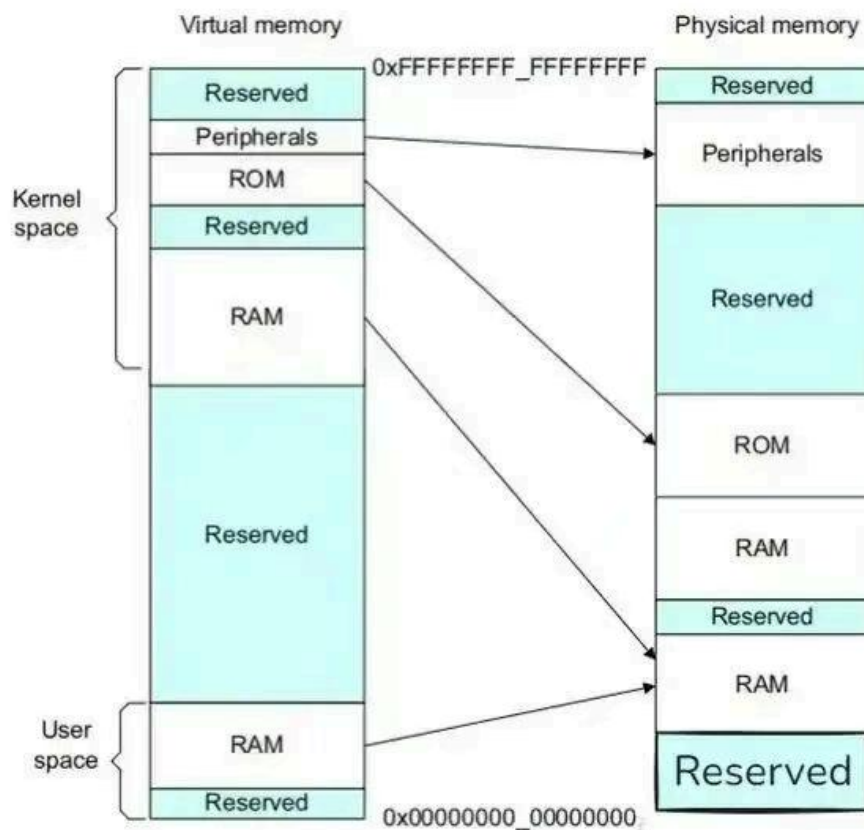
举个例子，假设我们有一个程序要访问虚拟地址 0x12345678。MMU 首先会提取出页号，比如是 0x1234，然后在页表中查找这个页号对应的条目。假设找到的条目显示对应的物理页号是 0x5678，而页内偏移是 0x9ABC，那么最终的物理地址就是 0x56789ABC。通过这样的转换，程序就能够顺利地访问到它所需要的数据，就好像我们通过地图上的路线规划找到了实际的目的地一样。

mmu开启以后会有以下特点：

1. 多个程序独立运行
2. 虚拟地址是连续的（物理内存可以有碎片）

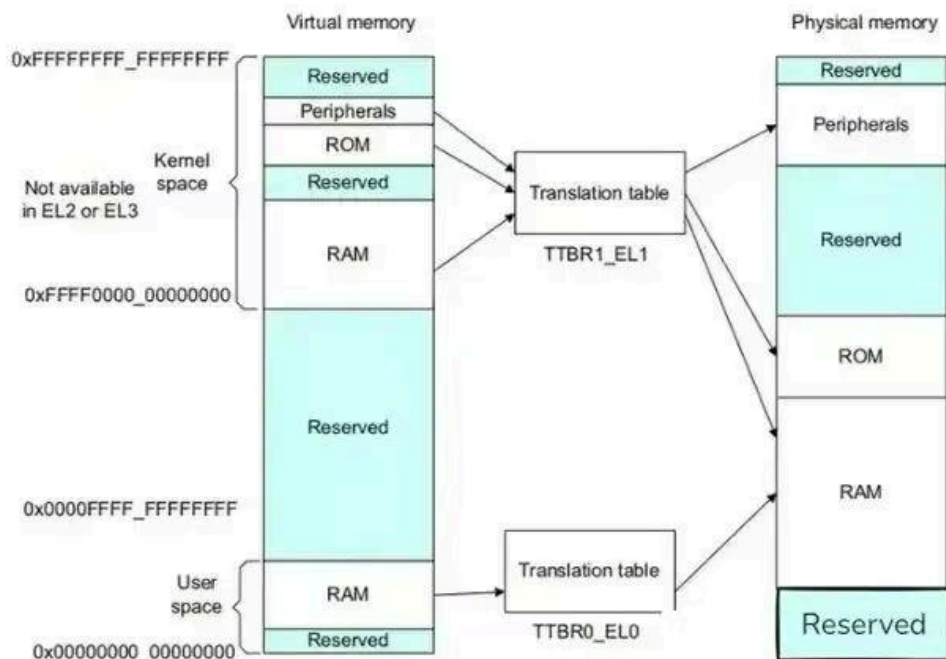
3. 允许操作系统管理内存

下图显示的系统说明了内存的虚拟和物理视图。单个系统中的不同处理器和设备可能具有不同的虚拟地址映射和物理地址映射。操作系统编写程序，使MMU在这两个内存视图之间进行转换：



要做到这一点，虚拟内存系统中的硬件必须提供地址转换，即将处理器发出的虚拟地址转换为主内存中的物理地址。MMU使用虚拟地址中最重要的位来索引转换表中的条目，并确定正在访问哪个块。MMU将代码和数据的虚拟地址转换为实际系统中的物理地址。该转换将在硬件中自动执行，并且对应用程序是透明的。除了地址转换之外，MMU还可以控制每个内存区域的内存访问权限、内存顺序和缓存策略。

MMU对执行的任务或应用程序可以不了解系统的物理内存映射，也可以不了解同时运行的其他程序。每个程序可以使用相同的虚拟内存地址空间。即使物理内存是碎片化的，还可以使用一个连续的虚拟内存映射。此虚拟地址空间与系统中内存的实际物理映射分开的。应用程序被编写、编译和链接，以在虚拟内存空间中运行。



如上图所示，TLB是MMU中最近访问的页面翻译的缓存。对于处理器执行的每个内存访问，MMU将检查转换是否缓存在TLB中。如果所请求的地址转换在TLB中导致命中，则该地址的翻译立即可用。TLB本质是一块高速缓存。数据cache缓存地址(虚拟地址或者物理地址)和数据。TLB缓存虚拟地址和其映射的物理地址。TLB根据虚拟地址查找cache，它没得选，只能根据虚拟地址查找。所以TLB是一个虚拟高速缓存。

每个TLB entry通常不仅包含物理地址和虚拟地址，还包含诸如内存类型、缓存策略、访问权限、地址空间ID (ASID) 和虚拟机ID (VMID) 等属性。如果TLB不包含处理器发出的虚拟地址的有效转换，称为TLB Miss，则将执行外部转换页表查找。MMU内的专用硬件使它能够读取内存中的转换表。

然后，如果翻译页表没有导致页面故障，则可以将新加载的翻译缓存在TLB中，以便进行后续的重用。简单概括一下就是：硬件存在TLB后，虚拟地址到物理地址的转换过程发生了变化。虚拟地址首先发往TLB确认是否命中cache，如果cache hit直接可以得到物理地址。否则，一级一级查找页表获取物理地址。并将虚拟地址和物理地址的映射关系缓存到TLB中。

如果操作系统修改了可能已经缓存在TLB中的转换的entry，那么操作系统就有责任使这些未更新的TLB entry invalid。当执行A64代码时，有一个TLBI，它是一个TLB无效的指令：

```
TLBI <type><level>{IS} {, <Xt>}
```

TLB可以保存固定数量的entry。可以通过由转换页表遍历引起的外部内存访问次数和获得高TLB命中率来获得最佳性能。ARMv8-A体系结构 提供了一个被称为连续块entry的特性，以有效地利用TLB空间。转换表每个entry都包含一个连续的位。当设置时，这个位向TLB发出信号，表明它可以缓存一个覆盖多个块转换的单个entry。查找可以索引到连续块所覆盖的地址范围中的任何位置。因此，TLB可以为已定义的范围缓存一个entry从而可以在TLB中存储更大范围的虚拟地址。

②内存保护的坚固防线

MMU 不仅是地址转换的高手，还是内存保护的坚固防线。在多任务的操作系统环境下，多个进程同时运行，就像一个热闹的大社区里住着许多户人家。如果没有有效的管理，这些进程可能会互相干扰，就像邻居之间随意闯入对方的房子一样。

MMU 通过硬件机制实现了内存访问授权，为每个进程分配了独立的地址空间，就像给每个家庭都分配了独立的房子，互不干扰。当一个进程试图访问内存时，MMU 会检查该进程是否有权限访问目标内存区域。只有当进程具有相应的访问权限时，MMU 才会允许这次访问，否则就会阻止访问，并向操作系统报告，就像保安会阻止没有权限的人进入特定区域一样。

例如，一个进程 A 正在运行，它只能访问属于自己地址空间内的内存。如果进程 A 试图访问进程 B 的内存区域，MMU 会立即发现这个非法访问行为，并阻止它，从而保证了系统的稳定性和安全性。这种内存保护机制有效地防止了进程之间的非法访问，避免了因一个进程的错误而导致整个系统崩溃的情况，就像坚固的围墙保护着每个家庭的安全，让整个社区能够和谐稳定地运转。

1.2页表：地址映射的导航图

页表是内存管理中不可或缺的部分，它就像是一本详细的导航图，记录着虚拟地址与物理地址之间的映射关系。在 32 位系统中，若页面大小为 4KB，虚拟地址空间为 4GB，那么页表可能会有 1048576 (2^{20}) 个表项，每个表项对应一个 4KB 的页面。为了节省内存空间，现代操作系统普遍采用多级页表结构。以二级页表为例，虚拟地址被进一步划分为外层页号、内层页号和页内偏移。外层页号用于索引一级页表（页目录），找到对应的二级页表起始地址；内层页号再用于索引二级页表，找到对应的物理页框号；最后，物理页框号与页内偏移组合得到物理地址。这样，对于那些未使用的虚拟地址空间，就不需要为其分配实际的页表项，大大节省了内存开销。

①在文件的读写中。在普通进程的匿名映射区操作中。在文件的读写中，进程对一个文件发起读请求，如果没哟对应的物理内存页，则内核处理程序首先在页缓存 中查找，如果找到直接返回。如果没找到，那么再分配空间，把内容从文件读入内存，插入页缓存，并返回，这里读操作虽然是一个页面发生的未命中，但是很大程度上会读取多个连续页面到内存中，只不过返回调用指定页面，因为实际情况中，很少仅仅需要读取一个页面，这样后续如果在需要访问还是需要进行磁盘IO，磁盘寻道时间往往比读取时间还要长，为了提高IO效率，就有了上述的预读机制。。

②在普通进程的匿名映射中，比如进程申请的比较大的堆空间（大于128kb的都是利用MMAP映射的），这些空间的内容如果在一段时间内没被释放 且在一段时间内没被访问，则有可能被系统内存管理器交换到外存以腾出更多的物理内存空间。下次在进程访问该内容，根据页表查找发现物理页面不在内存，则需要调用缺页异常处理程序。异常处理程序判断该页时被换出到外存，那么首先也是在页缓存中找，如果找到就返回；否则，从交换分区 或者交换文件中把对应的文件读入内存。这里所说的页缓存和1中描述的页缓存是一类缓存，但是并不是一个，1中的页缓存是文件系统相关的，在磁盘上对应着具体的文件；在页面回收的时候需要回写到磁盘即同步一下；而2中的页缓存其实是一种交换缓存，在页面回收的时候只能是换出到交换分区或者交换文件，不存在同步的操作。

(1) Linux内核页表结构

在Linux内核中，页表是用来管理虚拟内存和物理内存之间映射关系的数据结构。Linux采用了两级页表（two-level page table）的结构，具体包括以下几个部分：

1. 一级页表（Level 1 Page Table）：也称为顶层页表（Top-Level Page Table），它是整个页表层次结构的入口点。一级页表的条目数与物理内存大小相关，每个条目指向二级页表。
2. 二级页表（Level 2 Page Table）：也称为中间层页表（Intermediate Page Table）。二级页表将虚拟地址空间划分成更小的区域，并对应到三级页表。
3. 三级页表（Level 3 Page Table）：也称为底层页表（Leaf Page Table）。三级页表对应着最细粒度的页面大小，通常是4KB或2MB。每个条目包含了虚拟地址和物理地址之间的映射关系。

通过多级页表结构，Linux 内核能够有效地管理大型虚拟地址空间和物理内存。当进程访问一个虚拟地址时，内核会根据虚拟地址在不同层次的页表中进行查找和转换，最终得到对应的物理地址。

广告

中国广电流量卡

19元192G/月

☑ 首月免租 超值资费 ☑ 无漫游 全国免接听
☑ 海量靓号 免费任选 ☑ 免费领取 包邮到家

该号卡由中国广电提供服务，资费说明详见广告内，具体号码以实际领取为准



话费贵、流量少，你要领它！月租19元192G/月通用，首月0月租！

 中国广电直营店

[查看](#)

需要注意的是，在64位版本的 Linux 内核中，由于更广阔的虚拟地址空间和更大容量的物理内存支持，可能会有更多级的页表结构。具体的层次结构和细节可能因不同的架构、配置和内核版本而有所变化，上述描述仅为一般情况下的常见页表结构。

①页表项

页表项（Page Table Entry）是页表中的每个条目，用于描述虚拟地址与物理地址之间的映射关系。每个页表项通常包含以下字段：

1. 物理页面基址（Physical Page Base Address）：指向对应的物理内存页面的起始地址。
2. 标志位（Flags）：用于控制页面的访问权限、缓存策略和其他特性。常见的标志位包括可读/写/执行权限、缓存禁用、全局标志等。
3. 状态位（Status Bits）：记录页面的状态信息，如脏页标记、引用位等。
4. 其他辅助字段：可能包含一些额外信息，如页表层次索引、保留位等。

根据操作系统和硬件架构的不同，页表项可能有不同的结构和长度。例如，在x86体系结构上，一个典型的32位页表项长度为4字节（32 bits），而在64位上则是8字节（64 bits）。这些长度限制了能够表示的物理内存大小和虚拟地址空间范围。

通过逐级查找和解析页表项，操作系统可以将虚拟地址转换为物理地址，并进行访问控制和管理。因此，正确配置和使用页表项对于有效管理内存和实现安全性至关重要。

②各种类型的页面

- Page Table Entry (PTE): 记录了一个页面在物理内存中的地址以及相关信息，用于虚拟地址到物理地址的映射。
- Page Global Directory (PGD): 一种高级页表结构，用于管理大量虚拟内存。它将多个Page Table组织起来，形成更大的虚拟地址空间。
- Page Middle Directory (PMD): 类似于PGD，但粒度更小。通常用于管理特定区域内的虚拟内存。
- Inverted Page Table (IPT): 与传统页表不同，IPT根据物理地址索引每个页面对应的虚拟地址和进程信息。它可以减少页表占用的内存，并提高查找效率。
- Translation Lookaside Buffer (TLB): 高速缓存，位于CPU芯片上，加速虚拟地址到物理地址转换过程。

(2) 虚拟地址到物理地址的映射过程

虚拟地址到物理地址的映射过程是通过页表完成的。在使用虚拟内存时，操作系统将物理内存划分成固定大小的页面（通常为4KB），并将每个进程的虚拟地址空间划分成相同大小的页面。

当程序访问虚拟地址时，CPU会将该虚拟地址发送给内存管理单元（MMU），其中包含一个页表基址寄存器（Page Table Base Register, PTBR）指向当前进程的页表。

映射过程如下：

1. CPU将虚拟地址发送给MMU。
2. MMU根据虚拟地址中的页号找到对应的页表项。
3. 如果页表项中标记为有效，则获取对应的物理页框号。
4. 将物理页框号与虚拟地址中的偏移量组合，得到最终的物理地址。
5. 将最终物理地址返回给CPU，程序可以访问该物理地址上存储的数据。

如果在步骤3中发现页表项无效，则表示所请求的页面不在物理内存中。这时可能触发缺页异常，操作系统需要从磁盘上加载对应页面，并更新页表项以建立新的映射关系。

1.3高速缓存（Cache）：提升访问速度的加速器

高速缓存（Cache）是提高内存访问速度的关键硬件组件，它利用了程序访问的局部性原理，即程序在一段时间内往往会频繁访问相同或相邻的内存区域。Cache通常分为多级，如L1、L2和L3 Cache。L1 Cache速度最快，但容量最小，离CPU核心最近；L3 Cache容量最大，但速度相对较慢。当CPU需要访问数据时，首先会在L1 Cache中查找，如果命中（找到所需数据），则可以快速获取数据，大大缩短访问时间；如果未命中，则会依次到L2、L3 Cache以及主内存中查找。

例如，当CPU执行一个循环读取数组元素的操作时，由于数组元素在内存中是连续存储的，根据空间局部性原理，Cache会将该数组附近的内存块一并加载进来，这样在后续读取数组元素时，就很可能在Cache中命中，从而提高访问效率。Cache的替换算法（如最近最少使用LRU

算法) 决定了当 Cache 空间已满时, 哪些数据会被替换出去, 以保证 Cache 中始终存储着最常用的数据。

TLB是一种高速缓存, 内存管理硬件使用它来改善虚拟地址到物理地址的转换速度。当前所有的个人桌面, 笔记本和服务处理器都使用TLB来进行虚拟地址到物理地址的映射。使用TLB内核可以快速的找到虚拟地址指向物理地址, 而不需要请求RAM内存获取虚拟地址到物理地址的映射关系。这与data cache和instruction caches有很大的相似之处。

TLB的原理如下:

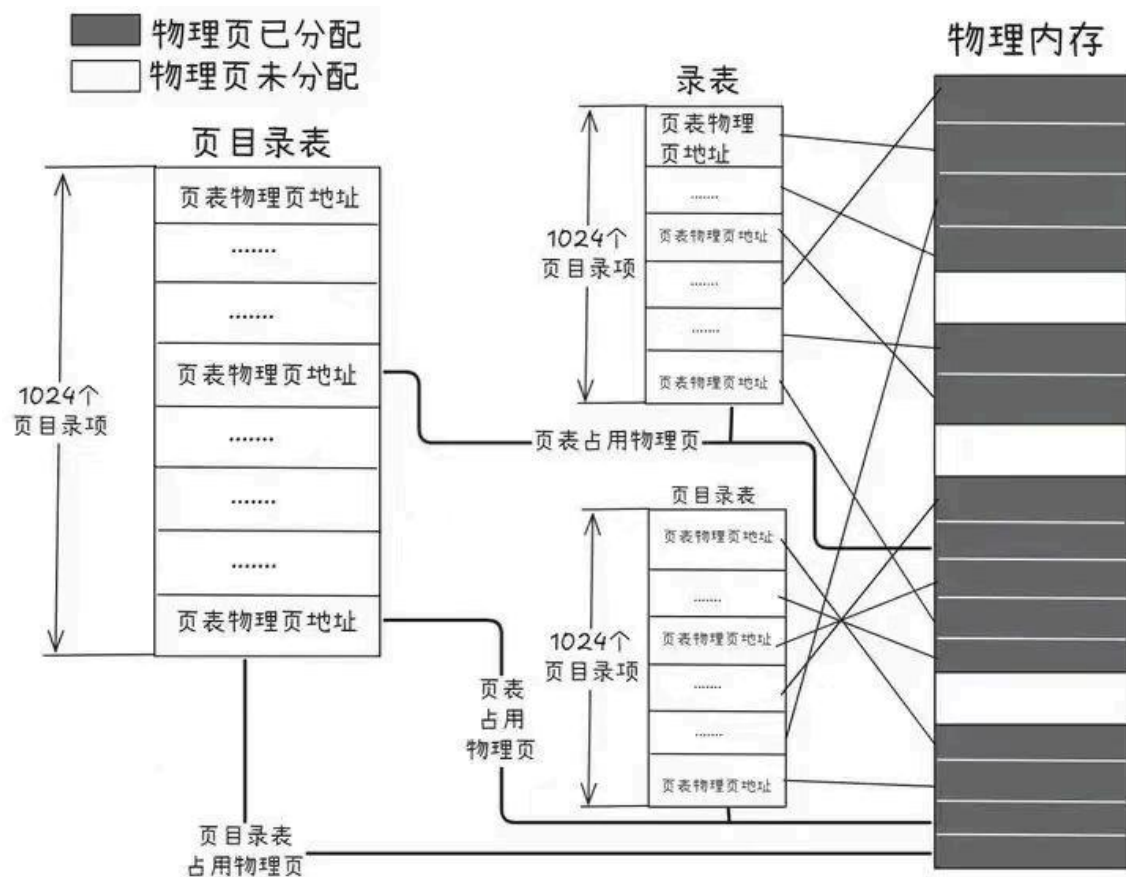
1. 当CPU访问一个虚拟地址时, 首先检查TLB中是否有对应的页表项。
2. 如果TLB中有对应的页表项(即命中), 则直接从TLB获取物理地址。
3. 如果TLB中没有对应的页表项(即未命中), 则需要访问内存来获取正确的页表项。
4. 在未命中情况下, 操作系统会进行相应处理, 从主存中获取正确的页表项, 并将其加载到TLB中以供后续使用。
5. 一旦正确的页表项加载到TLB中, CPU再次访问相同虚拟地址时就可以直接在TLB中找到映射关系, 提高了转换效率。

TLB具有快速查找和高效缓存机制, 能够极大地减少查询页表所需的时间。然而, 由于TLB是有限容量的, 在大型程序或多任务环境下可能无法完全覆盖所有需要转换的页面。当发生TLB未命中时, 则会导致额外的内存访问开销; 操作系统会负责管理和维护TLB, 包括缓存策略、TLB的刷新机制等。常见的缓存策略有全相联、组相联和直接映射等。

二、内存管理的硬件策略

2.1 分页机制：化整为零的智慧

在 Linux 的世界里, 内存分页机制就像是一位有条不紊的大管家, 精心管理着系统的内存资源。简单来说, 内存分页机制就是把物理内存和虚拟内存分割成固定大小的小块, 这些小块被称作“页”, 每个页的大小一般为 4KB 或者 8KB。就好比你要有一个巨大的仓库(内存), 为了更好地管理里面的货物(数据), 你把仓库划分成了一个大小相同的小隔间(页)。



分页机制是 80x86 内存管理机制的第二部分。它在分段机制的基础上完成虚拟地址到物理地址的转换过程。分段机制把逻辑地址转换成线性地址，而分页机制则把线性地址转换成物理地址。分页机制可用于任何一种分段模型。处理器分页机制会把线性地址空间划分成页面，然后这些线性地址空间页面被映射到物理地址空间的页面上。分页机制的几种页面级保护措施，可和分段机制保护措施或用或替代分段机制的保护措施。

(1) 分页机制如何启用

在我们进行程序开发的时候，一般情况下，是不需要管理内存的，也不需要操心内存够不够用，其实，这就是分页机制给我们带来的好处。它是实现虚拟存储的关键，位于线性地址与物理地址之间，在使用这种内存分页管理方法时，每个执行中的进程（任务）可以使用比实际内存容量大得多的连续地址空间。而且当系统内存实际上被分成很多凌乱的块时，它可以建立一个大而连续的内存空间的映象，好让程序不用操心和管理这些分散的内存块。分页机制增强了分段机制的性能。页地址变换是建立在段变换基础之上的。因为，段管理机制对于 Intel 处理器来说是最基本的，任何时候都无法关闭。所以即使启用了页管理功能，分段机制依然是起作用的，段部件也依然工作。

分页只能在保护模式（CR0.PE = 1）下使用。在保护模式下，是否开启分页，由 CR0.PG 位（位 31）决定：

- 当 CR0.PG = 0 时，未开启分页，线性地址等同于物理地址；
- 当 CR0.PG = 1 时，开启分页。

(2) 分页机制线性地址到物理地址转换过程

80x86 使用 4K 字节固定大小的页面，每个页面均是 4KB，并且对于 4K 地址边界处。这表示分页机制把 2^{32} 字节 (4GB) 的线性地址空间划分成 2^{20} (1M = 1048576) 个页面。分页机制通

过把线性地址空间中的页面重新定位到物理地址空间中进行操作。由于 4K 大小的页面作为一个单元进行映射，并且对于 4K 边界，因此线性地址的低 12 位可做为页内偏移量直接作为物理地址的低 12 位。分页机制执行的重定向功能可以看作是线性地址的高 20 位转换到对应物理地址的高 20 位。

线性到物理地址转换功能，被扩展成允许一个线性地址被标注为无效的，而非要让其产生一个物理地址。以下两种情况一个页面可以被标注为无效的：

- 1. 操作系统不支持的线性地址。
- 2. 对应的虚拟内存系统中的页面在磁盘上而非在物理内存中。

在第一中情况下，产生无效地址的程序必须被终止，在第二种情况下，该无效地址实际上是请求操作系统虚拟内存管理器把对应的页面从磁盘加载到物理内存中，以供程序访问。因为无效页面通常与虚拟存储系统相关，因此它们被称为不存在页面，由页表中称为存在的属性来确定。

当使用分页时，处理器会把线性地址空间划分成固定大小的页面（4KB），这些页面可以映射到物理内存中或磁盘存储空间中，当一个程序引用内存中的逻辑地址时，处理器会把该逻辑地址转换成一个线性地址，然后使用分页机制把该线性地址转换成对应的物理地址。

如果包含线性地址的页面不在当前物理内存中，处理器就会产生一个页错误异常。页错误异常处理程序就会让操作系统从磁盘中把相应页面加载到物理内存中（操作过程中可能会把物理内存中不同的页面写到磁盘上）。当页面加载到物理内存之后，从异常处理过程的返回操作会使异常的指令被重新执行。处理器把用于线性地址转换成物理地址和用于产生页错误的信息包含在存储与内存中的页目录与页表中。

2.2分段机制：各司其职的分工

分段机制则是根据程序的逻辑结构将内存划分为不同的段，如代码段、数据段、堆栈段等。每个段都有自己的段基址和段界限。逻辑地址由段号和段内偏移组成。当程序访问内存时，首先根据段号在段表中查找对应的段基址，然后将段基址与段内偏移相加，得到物理地址。例如，在 x86 架构中，CS 寄存器存储代码段的段选择子，DS 寄存器存储数据段的段选择子。

当执行一条指令时，会根据 CS 寄存器中的段选择子在段表中找到代码段的段基址，再加上指令中的偏移地址，从而确定指令在内存中的位置。分段机制的优点在于它更符合程序员的思维方式，便于程序的模块化设计和共享。比如，多个进程可以共享同一个代码段，减少内存的重复占用。然而，分段机制也存在一些局限性，如容易产生外部碎片，内存分配和管理相对复杂等。

（1）段选择子与段描述符

在理解了段的基本概念后，我们来看看段选择子和段描述符，它们在分段机制中起着至关重要的作用。

段选择子是一个 16 位的字段，它就像是一把钥匙，虽然不能直接打开段这个“房间”，但可以指向段描述符表中定义段的段描述符。段选择子包含三个部分：13 位的索引值，它就像是图书馆书架上的编号，用来指定段描述符在全局描述符表（GDT）或局部描述符表（LDT）中的位置；1 位的表指示标志 TI，TI = 0，表示描述符在 GDT 中，TI = 1，表示描述符在 LDT 中，这

就好比是告诉你要去哪个图书馆的书架找书；还有 2 位的请求特权级 RPL，它规定了访问该段所需的特权级别，就像是进入某些特殊房间需要特定的权限一样。

段描述符则是一个 8 字节的数据结构，它详细地描述了段的各种特征，就像是一份关于房子的详细说明书。段描述符中包含段基地址、段限长、段属性等重要信息。其中，段基地址明确了段的起始物理地址；段限长决定了段的长度；段属性则涵盖了段的访问权限（如可读、可写、可执行等）、特权级等特性。通过段描述符，CPU 可以全面了解一个段的情况，从而进行正确的内存访问。

(3) 逻辑地址到线性地址的转换

在 Linux 内核中，程序使用的是逻辑地址，而 CPU 访问内存时需要的是线性地址，因此就需要进行逻辑地址到线性地址的转换，这个转换过程是分段机制的核心。

当程序运行时，CPU 会遇到一个逻辑地址，它由段选择子和段内偏移量组成。为了将逻辑地址转换为线性地址，CPU 会按照以下步骤进行操作：

1. 查询 GDT/LDT 表：CPU 首先会从段选择子中提取出索引值和 TI 标志。如果 TI = 0，就表示要从全局描述符表 GDT 中查找段描述符；如果 TI = 1，则从局部描述符表 LDT 中查找。找到对应的描述符表后，根据索引值在表中定位到相应的段描述符，这就像是在图书馆的书架上找到了对应的书籍。
2. 检验访问权限和范围：找到段描述符后，CPU 会检查其中的段属性，确认当前的访问权限是否足够，以及偏移量是否在段限长的范围内。比如，如果当前程序试图写入一个只读的代码段，就会因为权限不足而触发异常；如果偏移量超出了段限长，也会被判定为非法访问，同样会触发异常，这就像是没有权限进入特殊房间或者超出了房间的活动范围。
3. 计算线性地址：在确认访问合法后，CPU 会将段描述符中的段基地址与逻辑地址中的段内偏移量相加，从而得到线性地址，这就像是找到了房间后，根据房间内的具体位置信息找到了对应的物品。这个线性地址就是 CPU 在下一步进行内存访问时所使用的地址。

2.3 虚拟内存技术：拓展内存的魔法

虚拟内存技术是利用磁盘空间来扩充物理内存的一种神奇技术。它使得程序可以使用比实际物理内存更大的内存空间。其原理是将物理内存和磁盘空间结合起来，操作系统将物理内存划分为固定大小的页框，将虚拟内存划分为同样大小的页面。当程序访问的页面不在物理内存中时，就会触发缺页中断。操作系统会根据一定的页置换算法（如最近最少使用 LRU 算法），选择一个物理内存中的页面将其数据写回磁盘，然后将程序需要的页面从磁盘加载到物理内存中。

广告

在线验证

报告语种

中文

英文

学历证书电子注册备案表



姓名

性别

出生日期

入学日期

毕(结)业日期

学校名称

专业

学制

层次

学历类别

学习形式

研耀教育

查看

四川在职研究生(招生): 2年制硕士2025招生开启, 无需到校上课!

研耀教育

例如，当一个程序运行时，它可能会申请大量的内存，但实际物理内存有限。此时，操作系统会将一些暂时不使用的页面（如程序中很久未访问的数据页面）交换到磁盘上的交换文件（Page File）中，当程序再次需要这些页面时，再从磁盘中读取回来。这样，程序就可以在有限的物理内存条件下运行，大大提高了系统的多任务处理能力和内存的使用效率。

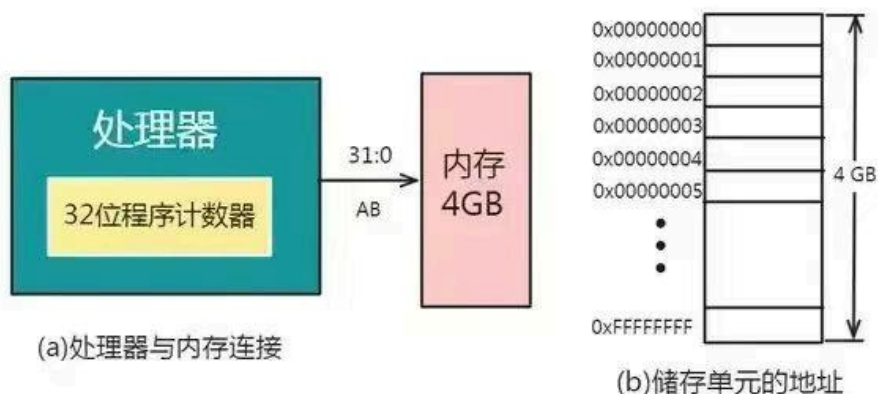
(1) 为什么需要使用虚拟内存

进程需要使用的代码和数据都放在内存中，比放在外存中要快很多。问题是内存空间太小了，不能满足进程的需求，而且现在都是多进程，情况更加糟糕。所以提出了虚拟内存，使得每个进程用于3G的独立用户内存空间和共享的1G内核内存空间。（每个进程都有自己的页表，才使得3G用户空间的独立）这样进程运行的速度必然很快了。而且虚拟内存机制还解决了内存碎片和内存不连续的问题。为什么可以在有限的物理内存上达到这样的效果呢？

为了更直观地理解，我们可以把虚拟内存想象成一个图书馆的目录系统。每个进程就像是一个读者，拥有自己的目录（虚拟地址空间）。当读者想要查找某本书（访问数据）时，会先在自己的目录中找到对应的条目（虚拟地址），然后通过这个条目去书架（物理内存）上找到实际的书。如果书架上没有这本书（缺页异常），图书馆管理员（操作系统）就会从仓库（磁盘）中把书取出来放到书架上，并更新目录（页表），以便下次读者能更快地找到这本书。

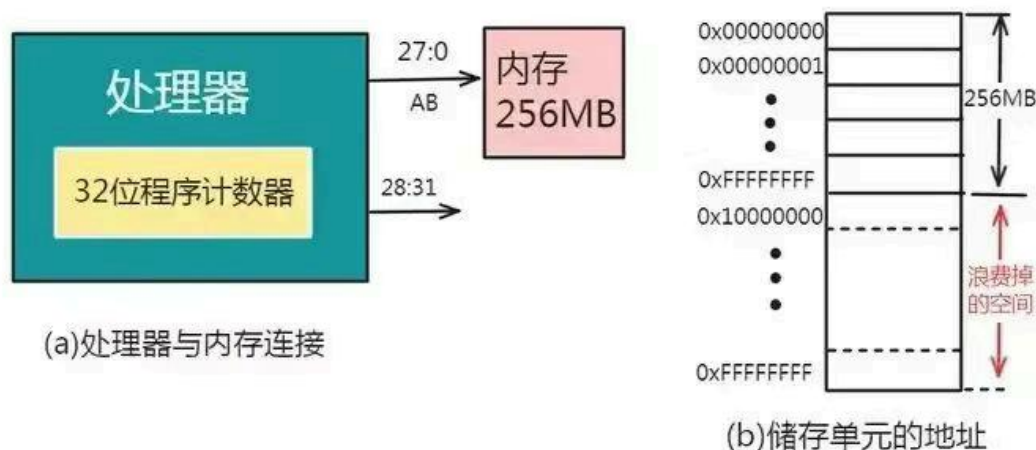
例如：对于程序计数器位数为32位的处理器来说，他的地址发生器所能发出的地址数目为 $2^{32}=4G$ 个，于是这个处理器所能访问的最大内存空间就是4G。在计算机技术中，这个值就叫做处理器的寻址空间或寻址能力。

照理说，为了充分利用处理器的寻址空间，就应按照处理器的最大寻址来为其分配系统的内存。如果处理器具有32位程序计数器，那么就应按照下图的方式，为其配备4G的内存：

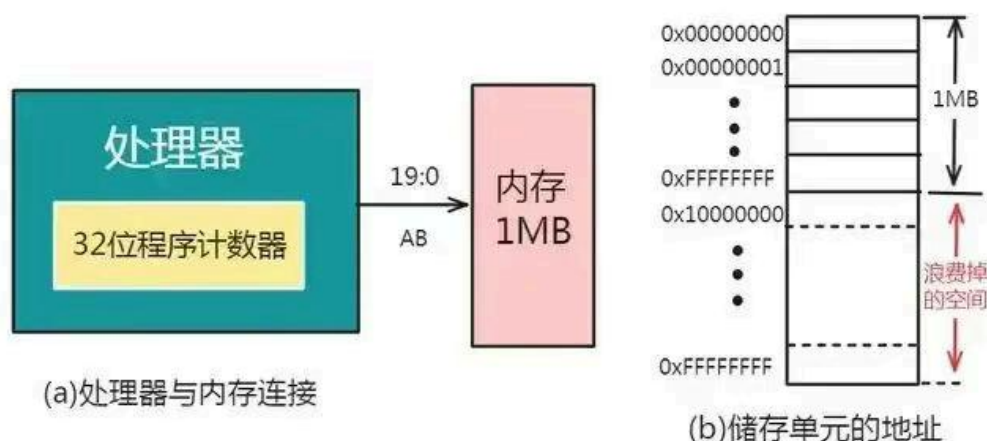


这样，处理器所发出的每一个地址都会有一个真实的物理存储单元与之对应；同时，每一个物理存储单元都有唯一的地址与之对应。这显然是一种最理想的情况。

但遗憾的是，实际上计算机所配置内存的实际空间常常小于处理器的寻址范围，这是就会因处理器的一部分寻址空间没有对应的物理存储单元，从而导致处理器寻址能力的浪费。例如：如下图的系统中，具有32位寻址能力的处理器只配置了256M的内存储器，这就会造成大量的浪费：



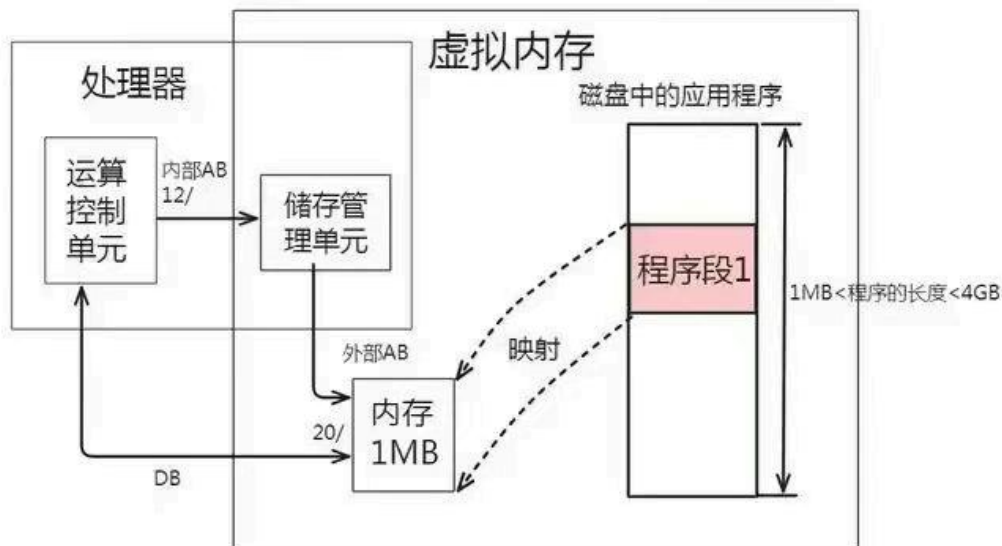
另外，还有一些处理器因外部地址线的根数小于处理器程序计数器的位数，而使地址总线的根数不满足处理器的寻址范围，从而处理器的其余寻址能力也就被浪费了。例如：Intel8086处理器的程序计数器位32位，而处理器芯片的外部地址总线只有20根，所以它所能配置的最大内存为1MB：



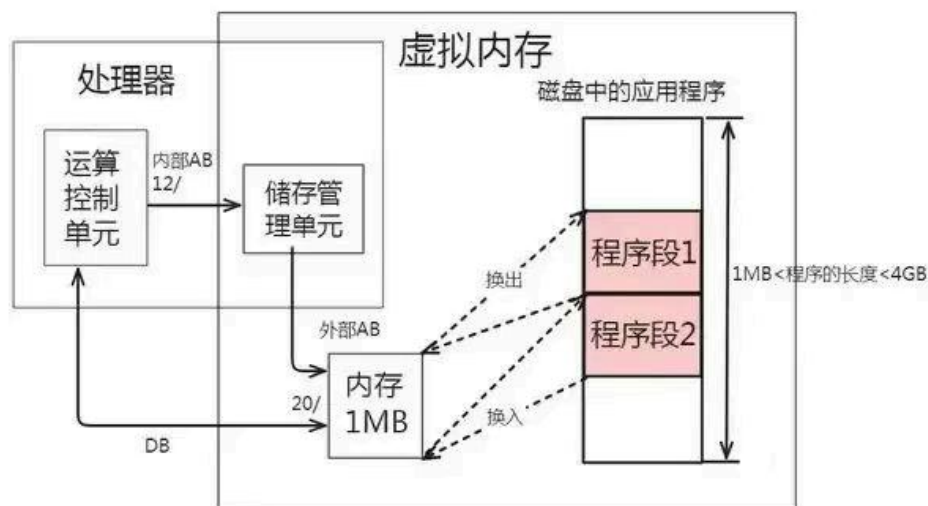
在实际的应用中，如果需要运行的应用程序比较小，所需内存容量小于计算机实际所配置的内存空间，自然不会出什么问题。但是，目前很多的应用程序都比较大，计算机实际所配置的内存空间无法满足。

实践和研究都证明：一个应用程序总是逐段被运行的，而且在一段时间内会稳定运行在某一段程序里。

这也就出现了一个方法：如下图所示，把要运行的那一段程序自辅存复制到内存中来运行，而其他暂时不运行的程序段就让它仍然留在辅存。



当需要执行另一端尚未在内存的程序段（如程序段2），如下图所示，就可以把内存中程序段1的副本复制回辅存，在内存腾出必要的空间后，再把辅存中的程序段2复制到内存空间来执行即可：



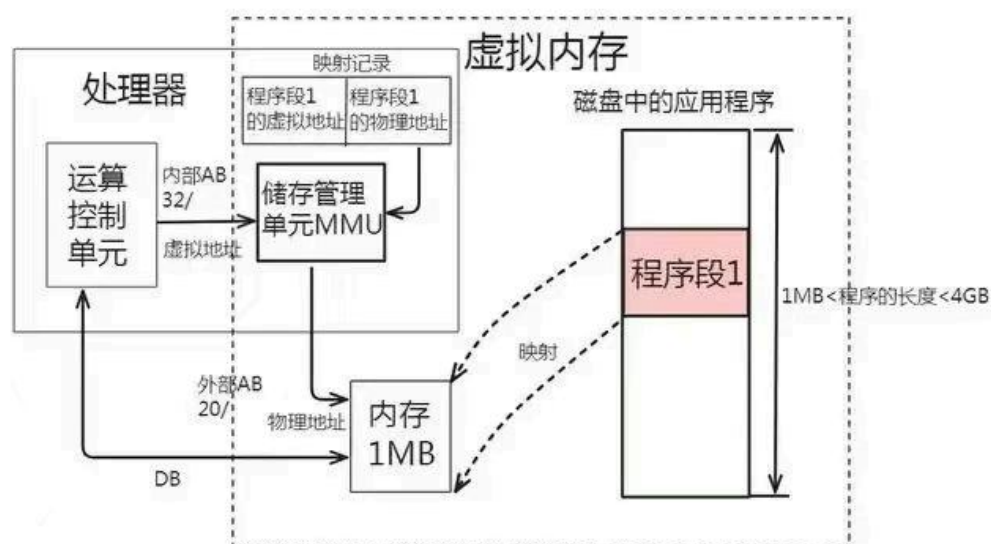
在计算机技术中，把内存中的程序段复制回辅存的做法叫做“换出”，而把辅存中程序段映射到内存的做法叫做“换入”。经过不断有目的的换入和换出，处理器就可以运行一个大于实际物理内存的应用程序了。或者说，处理器似乎是拥有了一个大于实际物理内存的内存空间。于是，这个存储空间叫做虚拟内存空间，而把真正的内存叫做实际物理内存，或简称为物理内存。

那么对于一台真实的计算机来说，它的虚拟内存空间又有多大呢？计算机虚拟内存空间的大小是由程序计数器的寻址能力来决定的。例如：在程序计数器的位数为32的处理器中，它的虚拟内存空间就为4GB。

可见，如果一个系统采用了虚拟内存技术，那么它就存在着两个内存空间：虚拟内存空间和物理内存空间。虚拟内存空间中的地址叫做“虚拟地址”；而实际物理内存空间中的地址叫做“实际物理地址”或“物理地址”。处理器运算器和应用程序设计人员看到的只是虚拟内存空间和虚拟地址，而处理器片外的地址总线看到的只是物理地址空间和物理地址。

由于存在两个内存地址，因此一个应用程序从编写到被执行，需要进行两次映射。第一次是映射到虚拟内存空间，第二次时映射到物理内存空间。在计算机系统中，第二次映射的工作是由硬件和软件共同来完成的。承担这个任务的硬件部分叫做存储管理单元MMU，软件部分就是操作系统的内存管理模块了。

在映射工作中，为了记录程序段占用物理内存的情况，操作系统的内存管理模块需要建立一个表格，该表格以虚拟地址为索引，记录了程序段所占用的物理内存的物理地址。这个虚拟地址/物理地址记录表便是存储管理单元MMU把虚拟地址转化为实际物理地址的依据，记录表与存储管理单元MMU的作用如下图所示：



综上所述，虚拟内存技术的实现，是建立在应用程序可以分成段，并且具有“在任何时候正在使用的信息总是所有存储信息的一小部分”的局部特性基础上的。它是通过用辅存空间模拟RAM来实现的一种使机器的作业地址空间大于实际内存的技术。

从处理器运算装置和程序设计人员的角度来看，它面对的是一个用MMU、映射记录表和物理内存封装起来的一个虚拟内存空间，这个存储空间的大小取决于处理器程序计数器的寻址空间。

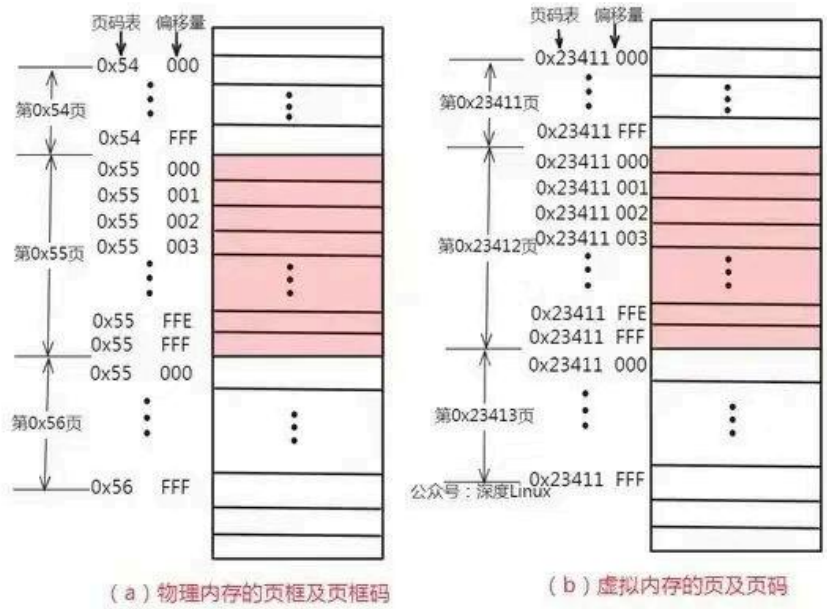
可见，程序映射表是实现虚拟内存的技术关键，它可给系统带来如下特点：

- 系统中每一个程序各自都有一个大小与处理器寻址空间相等的虚拟内存空间；
- 在一个具体时刻，处理器只能使用其中一个程序的映射记录表，因此它只看到多个程序虚存空间中的一个，这样就保证了各个程序的虚存空间时互不相扰、各自独立的；
- 使用程序映射表可方便地实现物理内存的共享。

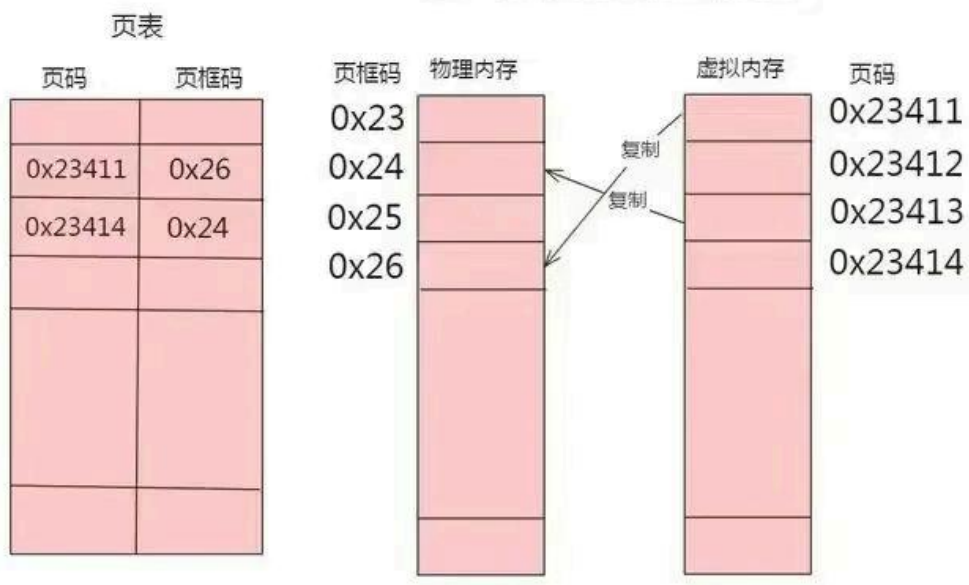
(2) 虚拟内存的页、物理内存的页框及页表

在Linux中，页与页框的大小一般为4KB。当然，根据系统和应用的不同，页与页框的大小也可有所变化。

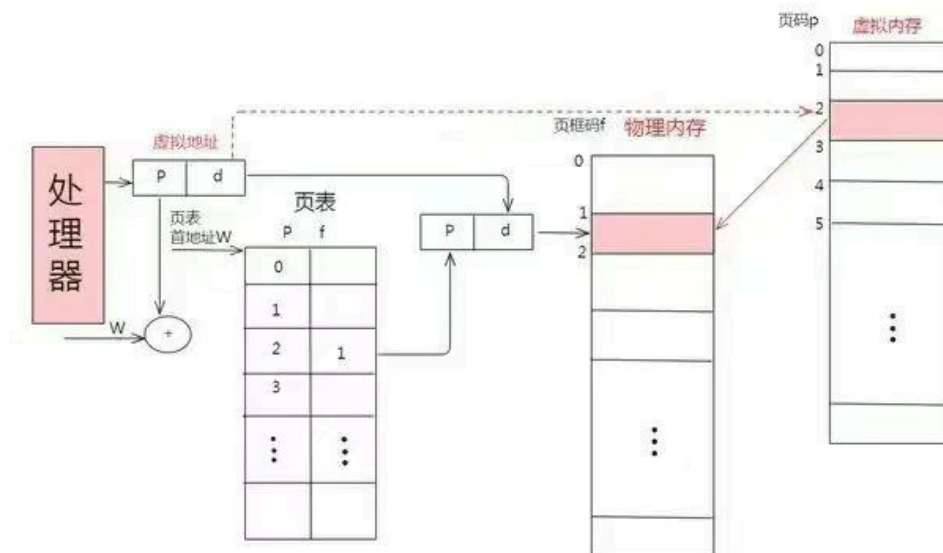
物理内存和虚拟内存被分成了页框与页之后，其存储单元原来的地址都被自然地分成了两段，并且这两段各自代表着不同的意义：高位段分别叫做页框码和页码，它们是识别页框和页的编码；低位段分别叫做页框偏移量和页内偏移量，它们是存储单元在页框和页内的地址编码。下图就是两段虚拟内存和物理内存分页之后的情况：



为了使系统可以正确的访问虚存页在对应页框中的映像，在把一个页映射到某个页框上的同时，就必须把页码和存放该页映像的页框码填入一个叫做页表的表项中。**这个页表就是之前提到的映射记录表。一个页表的示意图如下所示：**



页模式下，虚拟地址、物理地址转换关系的示意图如下所示：



也就是说：处理器遇到的地址都是虚拟地址。虚拟地址和物理地址都分成页码（页框码）和偏移值两部分。在由虚拟地址转化成物理地址的过程中，偏移值不变。而页码和页框码之间的映射就在一个映射记录表——页表中。

话说回来，内存映射是 Linux 中一种重要的内存管理技术，它允许将一个文件或者其他对象映射到进程的虚拟地址空间中，使得进程可以像访问内存一样直接访问文件。这种技术的核心优势在于提高了文件访问的效率，减少了内核和用户空间之间的数据拷贝。在 Linux 中，内存映射主要通过 `mmap()` 系统调用实现。

`mmap()` 函数将文件或其他对象映射到虚拟地址空间的一个连续区域，返回一个指向映射区域开始地址的指针。对该指针进行读写操作，实际上就是在访问文件内容。使用 `munmap()` 函数可以解除内存映射。

三、硬件内存管理的性能优化

3.1 硬件缓存优化：突破速度瓶颈

随着计算机技术的飞速发展，硬件缓存优化成为提升内存管理性能的关键方向。在多核处理器和多通道内存控制器的协同作用下，缓存的并行性得到了极大提升。以英特尔的酷睿 i9 系列处理器为例，其多核架构使得多个核心可以同时访问缓存，配合多通道内存控制器，能够显著提高数据的读写速度。当一个大型数据库应用程序运行时，多核处理器的不同核心可以同时从缓存中读取数据，多通道内存控制器则确保数据能够快速地在内存和缓存之间传输，从而加速数据库的查询操作。

硬件压缩技术也为缓存优化带来了新的思路。通过在硬件层面实现数据压缩和解压缩，能够有效减少内存占用，提高缓存的利用率。一些高端服务器采用了基于硬件的压缩技术，如英特尔的 QuickAssist Technology (QAT)，可以对存储在缓存中的数据进行实时压缩。当数据被写入缓存时，硬件压缩模块会自动对数据进行压缩，从而节省缓存空间；当数据被读取时，再进行快速解压缩。这在处理大规模数据时，能够显著提高缓存的命中率，减少对主内存的访问次数，进而提升系统性能。

新型非易失性内存（NVM）的出现，为缓存性能的提升提供了新的可能。相变存储器（PCM）、磁阻随机存取存储器（MRAM）等新型 NVM 具有非易失性、高存储密度和低功耗等优点。将这些新型 NVM 应用于缓存中，可以实现更快的数据访问速度和更高的缓存容量。例如，三星的基于 PCM 的缓存技术，在断电时数据不会丢失，并且能够快速响应读写请求，大大提高了系统的稳定性和性能。

3.2大页内存管理：提升效率的新途径

大页内存管理作为一种新兴的内存管理技术，正逐渐在高性能计算领域崭露头角。其原理是使用更大的页大小（如 2MB 或 1GB）来替代传统的 4KB 页大小，从而减少页表项的数量，优化内存访问性能。以数据库管理系统为例，如 Oracle 数据库，在处理大规模数据时，使用大页内存可以显著减少页表的开销。由于大页内存的页表项数量大幅减少，数据库在进行数据查询时，地址转换的时间大大缩短，从而提高了查询性能。在一个拥有 128GB 内存的数据库服务器中，使用 4KB 页大小可能会产生数百万个页表项，而使用 2MB 大页内存，页表项数量可以减少到原来的千分之一，这使得数据库的内存管理更加高效。

在虚拟化场景里，大页内存管理的价值同样不可忽视。以主流的 VMware ESXi 虚拟化平台为例，它对大页内存提供完善支持，这一特性直接降低虚拟机运行过程中的内存管理开销。要知道，虚拟机与宿主机之间的上下文切换，往往伴随着一系列复杂的内存管理操作，而大页内存可有效简化这些流程——通过减少上下文切换所需的时间，直接为虚拟机性能提升提供助力。尤其在运行多台虚拟机的服务器中，当每台虚拟机都启用大页内存后，不仅能显著减少内存碎片问题，让内存资源的利用率得到优化，还能从整体上进一步提升所有虚拟机的运行性能，为多负载虚拟化环境提供更稳定高效的内存支撑。

Linux内核 · 目录

上一篇

第一次编译内核就成功！——小白友好教程

下一篇

Linux设备驱动与 BSP：从硬件底层到软件实现

阅读 1568

广告

这个游戏实在太解压了！



跃动小子

立即试玩

